Forward Solver Codepath

Visual call graph for `solver_demo_slide15_no_speculative_cs.py`

PNPInverse / Forward solver

May 13, 2026

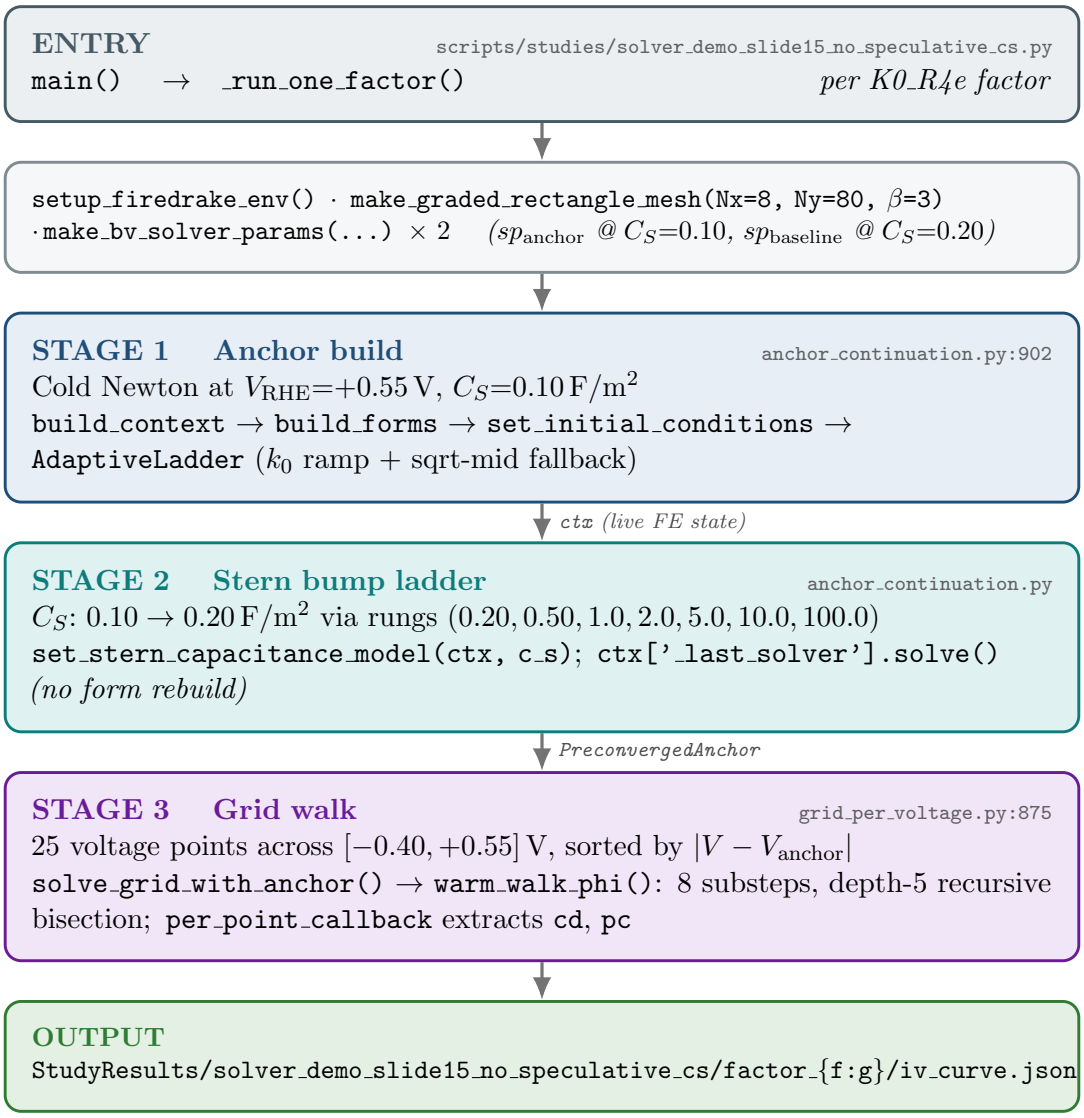
Abstract

Visual call graph for the code touched when the Cs^+/SO_4^{2-} baseline I-V demo runs — the dataset plotted by `plot_solver_demo_slide15_no_speculative_cs.py`. Color-coded by stage: **blue** anchor build, **teal** Stern bump, **purple** grid walk, **red** Newton/SNES core. Companion writeup *Derivation of the Analytic Bikerman Counterion Closure* (`analytic_counterion_derivation.pdf`) covers the math behind the Bikerman closure referenced as Layer 4.

---

1 Overview

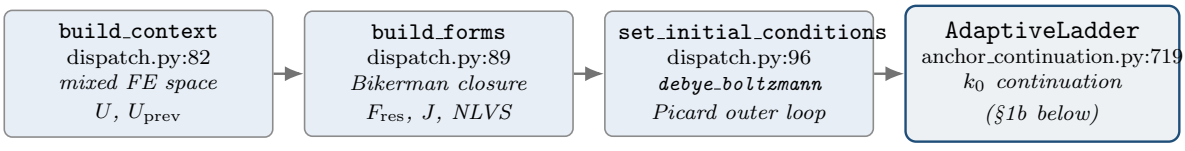
The demo's `main()` loops over four `K0_R4e` factors $\{1, 10^{-6}, 10^{-12}, 10^{-18}\}$, invoking `_run_one_factor()` for each. Each factor runs three stages on a shared mesh:



2 Stage 1 — Anchor build

The first solve in any run is cold: no warm-start is available. Stage 1 has a one-time setup (build context, build forms, install IC) followed by a k_0-scale continuation loop. Both parts must succeed for the anchor to converge.

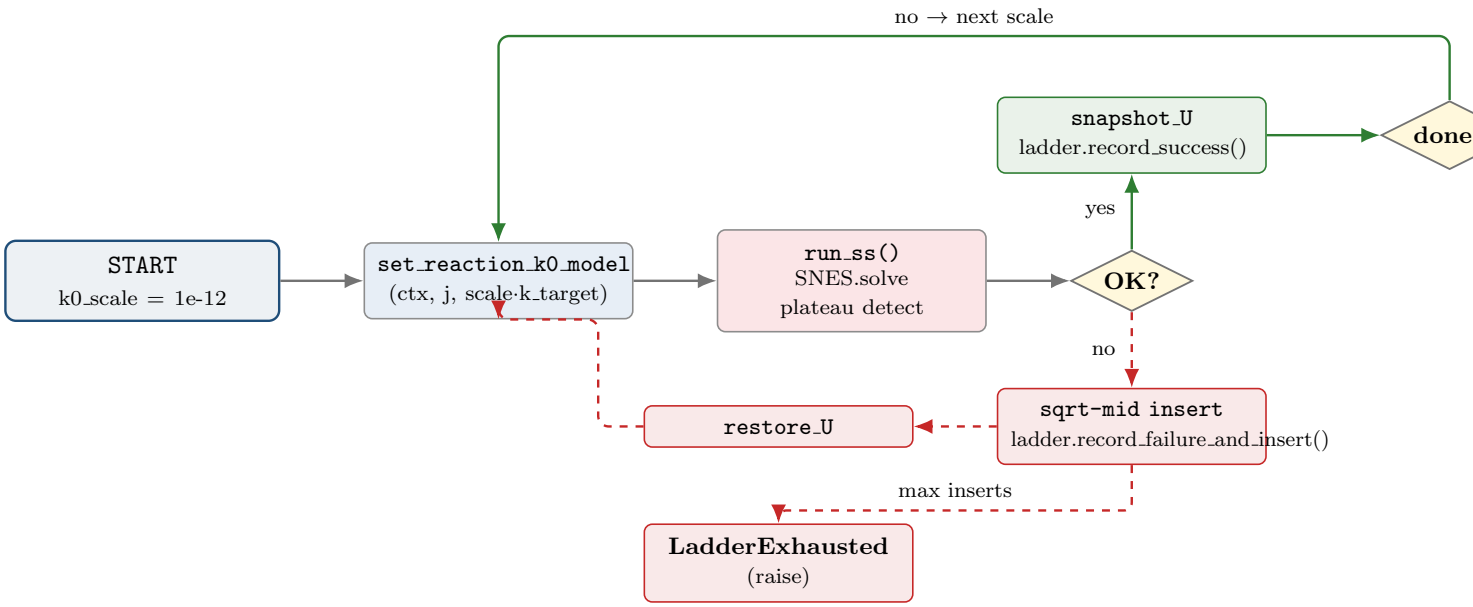
1a. One-time setup



Inside `build_forms`, the Bikerman counterion machinery is built by `build_steric_boltzmann_expressions` and `add_boltzmann_counterion_residual` in `boltzmann.py`. Inside `set_initial_conditions`, `picard_outer_loop_` in `picard_ic.py` runs a 2×2 scalar Picard on (ψ_S, ψ_D) to seed the EDL.

1b. Continuation loop (AdaptiveLadder)

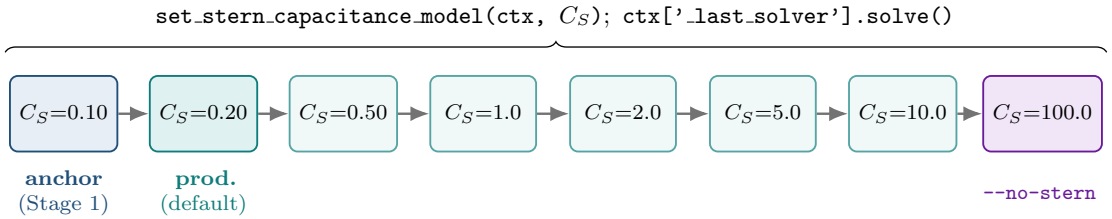
The loop ramps the kinetic prefactor k_0 from 10^{-12} (kinetics effectively off, diffusion-only Newton) up to the production value. On each rung it does a steady-state solve; on success it advances; on failure it inserts the geometric midpoint and retries:



After `max_inserts_per_step=4` consecutive failures, the loop raises `LadderExhausted`.

3 Stage 2 — Stern bump ladder

$C_S = 0.20\,\mathrm{F/m^2}$ won't converge cold, but $C_S = 0.10$ does. After Stage 1 lands at $C_S = 0.10$, ramp C_S upward through a verified rung sequence. Each rung mutates a shared FE `Constant` on `ctx` (no form rebuild) and reuses Stage 1's solver:



The bump-ladder helper truncates the verified rung list at the first rung ≥ target, so a $C_S = 0.20$ run lands in two steps and the no-Stern run uses all seven.

4 Stage 3 — Grid walk

The grid walk visits 25 voltage points sorted by distance from the anchor. Each point gets a fresh FE context built at its own V_{RHE} and is warm-started from its nearest already-converged neighbor. Voltage

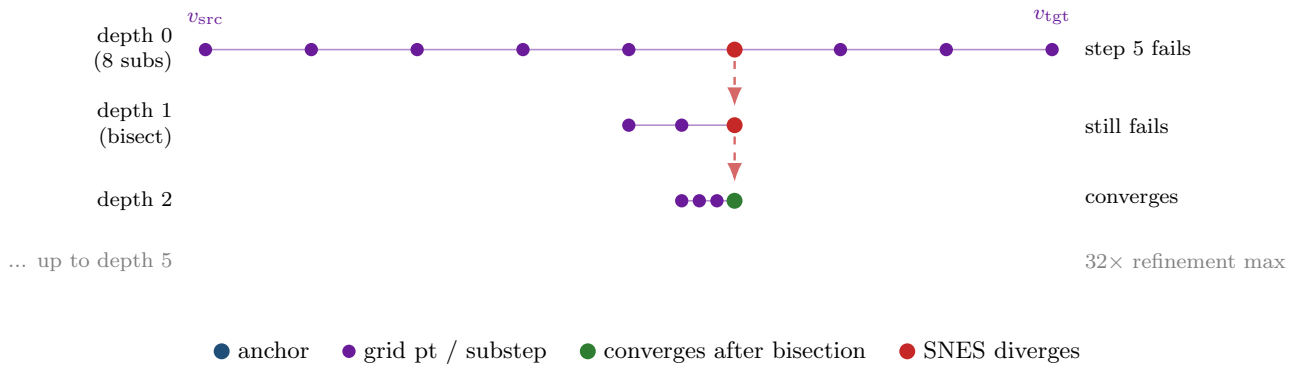
gaps are bridged by `warm_walk_phi`, which substeps and recursively bisects on failure.

3a. Grid layout and Frumkin cliff

The red dot at $V \approx 0\,\mathrm{V}$ is the Frumkin cliff — the voltage where Bikerman SO_4^{2-} saturation suddenly clamps the diffuse-layer charge balance. The walk visits the 24 non-anchor points in order of increasing $|V - V_{\mathrm{anchor}}|$, so anything past the cliff is warm-started from a neighbor still on the anodic side.

3b. One warm-walk step — substeps and recursive bisection

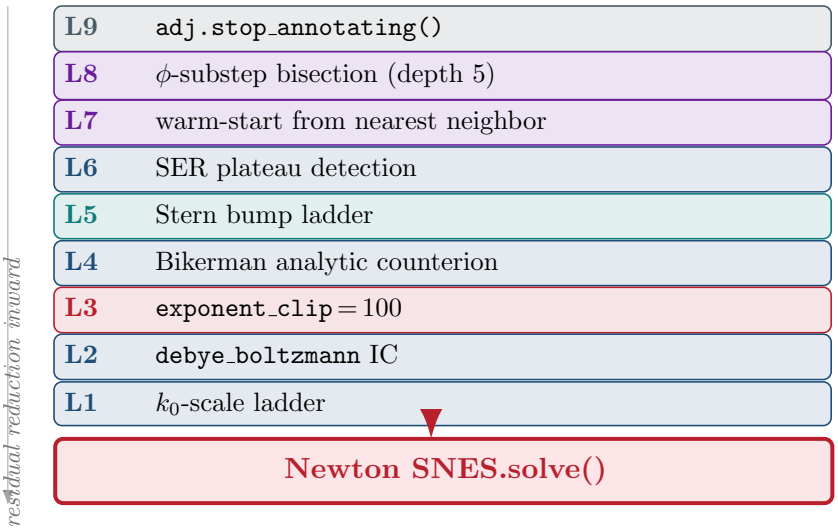
Between two consecutive visited points the walk substeps in 8 even increments; if Newton fails at any substep, the failing interval is halved and the walk recurses, up to depth 5 (32× refinement):



The Frumkin cliff at $V \approx 0\,\mathrm{V}$ is exactly where this recursive bisection earns its keep: substepping alone often isn't fine enough, and without depth-5 recursion the walk would mark several anodic-side points unconvergeable.

5 Layered defense — what keeps Newton in basin

The solver is defense in depth. Each layer below is one form of preconditioning around the innermost Newton/SNES solve — outer layers either reduce the residual the inner solve must handle, or recover when the inner solve fails.



L	Mechanism	Location	Why it matters
1	k_0-scale ladder ($10^{-12} \to 1.0$)	`AdaptiveLadder` @ `anchor_continuation.py:719`	Starts with kinetics off (Newton sees a diffusion-only problem), ramps to production k_0. Sqrt-midpoint insertion recovers from failed rungs without restart.
2	`debye_boltzmann` IC	`picard_ic.py` → `set_ic_debye_boltzmann_logc_muh`	Seeds the EDL with the right composite $\psi_S + \psi_D$ structure so Newton's initial residual is $\mathcal{O}(1)$ rather than $\mathcal{O}(10^{26})$.
3	`exponent_clip` = 100	`forms_logc_muh.py: _build_eta_clipped`	Clamps η_{scaled} before the $\alpha \cdot n_e$ multiplication. Without it $\exp(\alpha n_e \eta)$ over/underflows and SNES gets `Inf` in F.
4	Bikerman analytic counterion	`boltzmann.py:build_steric_boltzmann_expressions`	Cs^+ and SO_4^{2-} never enter the Newton state vector — they are closed-form functionals of ϕ. Eliminates transport modes that would otherwise stall Newton in the Debye layer.
5	Stern bump ladder	demo `_stern_bump_ladder` + `set_stern_capacitance_model`	$C_S = 0.20$ won't anchor cold; $C_S = 0.10$ will. Build at 0.10, ramp to 0.20 via verified rungs without form rebuild.
6	SER plateau detection	`make_run_ss` @ `grid_per_voltage.py:135`	Newton alone doesn't tell you BV is steady — you need $\int j\, ds$ to stop drifting. Plateau detector watches the surface flux and declares success when it stops moving.
7	Warm-start from nearest neighbor	`solve_grid_with_anchor` @ `grid_per_voltage.py:875`	Newton's basin radius in ϕ_{applied} is ~ 0.05 V; the anchor is up to 0.95 V from the cathodic edge, so the walk visits points in distance order, not list order.

L	Mechanism	Location	Why it matters
8	Recursive ϕ-substep bisection	`warm_walk_phi` @ `grid_per_voltage.py:218`	If the 8-substep warm walk fails between two grid points, the failing interval is halved and retried up to depth 5 ($32\times$ refinement). Catches the Frumkin cliff at $V \approx 0\,\mathrm{V}$.
9	`adj.stop_annotating()`	demo script around solver calls	Forward demo doesn't need the adjoint tape; wrapping prevents pyadjoint from recording 10^4+ ops per solve, which would OOM the process.

6 Files touched, by stage

Stage 1 — anchor build

- `anchor_continuation.py`: `solve_anchor_with_continuation`, `AdaptiveLadder`, `solve_reaction_k0_model`
- `dispatch.py`: `build_context`, `build_forms`, `set_initial_conditions`
- `forms_logc_muh.py`: `build_context_logc_muh`, `build_forms_logc_muh`, `set_ic_debye_boltzmann_logc_muh`, `_build_eta_clipped`
- `boltzmann.py`: `build_steric_boltzmann_expressions`, `add_boltzmann_counterion_residual`
- `picard_ic.py`, `multi_ion.py`, `mesh.py`, `nondim.py`

Stage 2 — Stern bump

- `anchor_continuation.py`: `set_stern_capacitance_model` (reuses Stage 1's `NonlinearVariationalSolver` stored on `ctx`)

Stage 3 — grid walk

- `grid_per_voltage.py`: `solve_grid_with_anchor`, `warm_walk_phi`, `_march` (bisection), `snapshot_U`, `restore_U`, `make_run_ss`
- `observables.py`: `_build_bv_observable_form`
- `dispatch.py`: same `build_context` / `build_forms` / `set_initial_conditions` as Stage 1, but invoked once per voltage on a fresh `ctx`

7 Per-factor wall time (typical)

Stage	Wall time	Dominant cost
Stage 1 anchor build	20–60 s	∼5 ladder rungs × several SS steps × Newton
Stage 2 Stern bump	10–30 s	7 rungs × 1 Newton solve each
Stage 3 grid walk	2–5 min	25 pts × warm-walk × ∼8 substeps × Newton
Total / factor	**3–7 min**	
4-factor sweep	**15–30 min**	

Stage 3 dominates wall time; the Frumkin cliff near $V \approx 0\,\mathrm{V}$ is the main source of variance because it triggers bisection events.

Notes

- The plot script `plot_solver_demo_slide15_no_speculative_cs.py` reads only the JSON files this codepath writes — no solver code is exercised by plotting.
- The same call graph holds for the no-Stern variant (`--no-stern`); the only differences are `stern_final` = 100.0 and the bump ladder reaching the rightmost (purple) rung in the Stage 2 diagram.
- This is *not* the inverse-solver codepath. The inverse path (`scripts/Inference/`) is paused and uses different orchestrators (`Inference/lm_tikhonov.py`, etc.).
- The Bikerman closure (Layer 4) is derived in detail in the companion writeup *Derivation of the Analytic Bikerman Counterion Closure* (`analytic_counterion_derivation.pdf`, this directory).